

История изменений

Дата	Версия	Описание	Автор
19.02.2026	1.1	Изменена структура документа и обновлено описание разделов	Кубарев Е.Д

Оглавление

1. Введение.....	3
2. Архитектурные цели и ограничения.....	4
3. Логическое представление.....	5
4. Представление развертывания.....	7
5. Представление данных, производительности и качества.....	8
6. Процессное представление.....	9
7. Сборка и развертывание.....	10
8. Представление реализации.....	11
9. Представление данных.....	12
10. Размер и производительность.....	13
11. Качество.....	14

1. Введение

Цель

Настоящий документ фиксирует архитектурный облик программного комплекса ALC (Assembly Line Control). Система предназначена для интеллектуального мониторинга и оперативного управления технологическими узлами в машиностроении. Документ служит основой для принятия технических решений, координации команды разработки и проведения код-ревью.

Область действия

Документ определяет архитектуру системы для сбора, обработки и визуализации данных с технологических узлов сборочных линий. В фокусе системы: сбор телеметрии, визуализация состояния в реальном времени на мобильных, веб и десктоп-платформах, оповещение операторов о событиях и оперативное управление параметрами. Вне фокуса системы: разработка прошивок для программируемых логических контроллеров (ПЛК), проектирование физического расположения датчиков.

Определения, сокращения и аббревиатуры

ALC — Assembly Line Control, программный комплекс контроля сборочных линий.

JWT — JSON Web Token, используется для аутентификации.

REST API — архитектурный стиль взаимодействия компонентов.

WebSocket / Socket.io — протокол и библиотека для двунаправленного обмена данными в реальном времени.

Аларм — сообщение о критическом событии или отклонении параметра.

Дашборд — информационная панель для визуализации данных.

2. Архитектурные цели и ограничения

Бизнес-цели

Кросс-платформенность: Доступ к системе для операторов с мобильных устройств (Android), через веб-браузеры и с настольных ПК (macOS).

Работа в реальном времени: Задержка отображения изменения состояния узлов не более 200 мс.

Масштабируемость: Поддержка одновременной работы до 500 операторов без потери производительности.

Надежность: Обеспечение целостности и сохранности журналов событий и телеметрии.

Технологический стек и ограничения:

Frontend:

React Native (Android)

React Native Web (браузер)

Electron (десктоп: macOS)

TypeScript

Socket.io-client

Axios

React Navigation

Redux Toolkit

Backend:

Node.js + Express

TypeScript

Socket.io

JWT

REST API

База данных: MongoDB.

Тестовые данные: Kaggle датасеты, скрипты конвертации на Node.js для загрузки в MongoDB.

3. Логическое представление

Система построена по классической клиент-серверной архитектуре. Клиентская часть (Frontend) реализована на единой кодовой базе и взаимодействует с серверной частью (Backend) через два основных канала: REST API для стандартных операций и WebSocket (Socket.io) для обмена данными в реальном времени.

Архитектурно значимые компоненты

Клиент:

Модуль навигации: React Navigation обеспечивает единообразную навигацию на всех целевых платформах.

Модуль управления состоянием: Redux Toolkit управляет глобальным состоянием приложения (данные узлов, пользовательская сессия).

Модуль API-клиента: Axios выполняет HTTP-запросы к REST API сервера.

Модуль real-time связи: Socket.io-client поддерживает постоянное соединение с сервером для приема обновлений и отправки команд.

Модуль визуализации: Отвечает за отображение графиков, дашбордов и состояний оборудования.

Сервер:

Модуль аутентификации: JWT-аутентификация, проверка учетных данных, управление доступом на основе ролей.

Модуль REST API: Обрабатывает запросы, не требующие real-time (исторические данные, управление пользователями).

Модуль WebSocket (Socket.io): Управляет подключениями клиентов, рассылает обновления и принимает команды.

Модуль бизнес-логики: Содержит правила обработки данных, проверки уставок и генерации событий.

Модуль доступа к данным: Обеспечивает взаимодействие с MongoDB.

Ключевые сценарии работы

1. Мониторинг: Клиент через Socket.io получает обновления состояния узлов. Сервер транслирует изменения параметров всем подключенным клиентам в реальном времени.

2. Оперативное управление: Инженер изменяет параметр через интерфейс. Frontend отправляет запрос на сервер (REST API). Сервер валидирует действие и обновляет состояние, оповещая всех клиентов об изменении через Socket.io.

3. Оповещение об алармах: При выходе параметра за допустимые пределы сервер генерирует событие, записывает его в БД и через Socket.io мгновенно уведомляет всех подписанных клиентов.

4. Представление развертывания

Система развертывается централизованно с клиентскими приложениями, устанавливаемыми на устройства пользователей.

Клиентский уровень:

Мобильное приложение (Android) распространяется посредством apk файлом.

Десктоп-приложения (macOS) собираются в инсталляционный пакет с помощью Electron.

Веб-приложение хостится на веб-сервере и доступно через браузер.

Серверный уровень:

Backend-приложение (Node.js + Express) развертывается на центральном сервере или в облаке.

Сервер открывает порты для HTTP (REST API) и WebSocket (Socket.io) соединений.

Уровень данных:

MongoDB развертывается на том же или выделенном сервере. Для обеспечения отказоустойчивости может использоваться репликация.

Сеть:

Все соединения между клиентами и сервером шифруются с использованием TLS. Аутентификация выполняется с помощью JWT-токенов.

5. Представление данных, производительности и качества

Наполнение тестовыми данными:

1. Загрузка CSV-файлов с промышленными данными с Kaggle.
2. Разработка скрипта Node.js для парсинга CSV, преобразования данных в структуру коллекций и загрузки в MongoDB. Скрипт также создает тестовых пользователей и генерирует синтетические события.

Производительность и масштабирование

Задержки: Доставка обновлений через WebSocket — не более 200 мс.

Время ответа REST API — не более 500 мс для типовых запросов.

Масштабирование: Backend может масштабироваться горизонтально за счет запуска нескольких экземпляров приложения за балансировщиком нагрузки. Socket.io поддерживает работу с несколькими серверами через специальные адаптеры.

Атрибуты качества

Надежность: Репликация MongoDB, автоматическое переподключение Socket.io, логирование критических операций.

Безопасность: JWT-аутентификация, хеширование паролей, шифрование трафика (TLS), разделение ролей (оператор, инженер, администратор), аудит действий.

Поддерживаемость: Единая кодовая база для всех клиентских платформ, документирование API (OpenAPI), централизованный сбор логов и метрик.

Производительность: Кэширование данных на клиенте (Redux Toolkit), оптимизация запросов к MongoDB (индексы), ленивая загрузка компонентов в React Native.

6. Процессное представление

Данные от технологических узлов поступают в систему .

Серверный модуль бизнес-логики валидирует и обогащает данные.

Данные записываются в соответствующие коллекции MongoDB.

Сервер через Socket.io транслирует обновления состояния всем подключенным клиентам, подписанным на соответствующие узлы.

Процесс управления узлами:

Оператор или инженер отправляет команду управления через интерфейс.

Frontend отправляет запрос через REST API на сервер.

Сервер аутентифицирует пользователя и проверяет его права.

Бизнес-логика обрабатывает команду и отправляет её на целевое устройство.

Результат выполнения команды записывается в БД, и клиент получает подтверждение через REST API или WebSocket.

Потеря связи с клиентом: Socket.io автоматически пытается переподключиться. В это время данные на сервере продолжают накапливаться. После восстановления соединения клиент получает актуальное состояние и пропущенные обновления.

Высокая нагрузка: При превышении пороговых значений нагрузки система должна масштабироваться горизонтально. В пиковых ситуациях возможно использование очередей сообщений для асинхронной обработки тяжелых операций.

7.Сборка и развертывание

Frontend:

Мобильные приложения: Сборка через EAS Build или стандартные инструменты React Native, публикация в App Store и Google Play.

Веб-приложение: Сборка в статические файлы, развертывание на любом веб-сервере.

Десктоп-приложение: Сборка Electron-приложения под каждую целевую платформу с использованием электрон-билдера.

Backend:

Контейнеризация приложения с помощью Docker.

Развертывание Docker-контейнеров в среде оркестрации или на виртуальных машинах.

Использование CI/CD для автоматизации сборки, тестирования и развертывания.

8. Представление реализации

Обзор:

Реализация системы организована в виде двух основных частей: клиентской и серверной, разрабатываемых независимо, но с соблюдением единых интерфейсов взаимодействия. Кодовая база разделена на логические модули с четкими границами ответственности.

Слои

Клиентский уровень

Уровень представления содержит React-компоненты пользовательского интерфейса, адаптированные для различных платформ с использованием возможностей React Native и React Native Web. Компоненты разделены на переиспользуемые элементы, экранные формы и контейнеры.

Уровень управления состоянием реализован с использованием Redux Toolkit и обеспечивает единое хранилище данных для всего приложения.

Стейт разделен на срезы, включая срез состояния узлов, срез пользовательской сессии, срез уведомлений и срез настроек.

Серверный уровень

Уровень контроллеров обрабатывает входящие HTTP-запросы к REST API, выполняет валидацию входных данных с использованием схем валидации и передает управление соответствующим сервисам. Контроллеры не содержат бизнес-логики, только маршрутизацию и обработку запросов.

Уровень сервисов содержит бизнес-логику приложения, включая обработку данных, проверку прав доступа, генерацию событий и координацию между различными модулями. Сервисы реализуют основные сценарии использования системы.

9. Представление данных

Хранение данных в системе организовано с использованием MongoDB как основного хранилища. Выбор MongoDB обусловлен необходимостью работы с полуструктурированными данными телеметрии, гибкостью схемы при добавлении новых типов узлов и параметров, а также хорошей масштабируемостью.

10. Размер и производительность

Объемы данных:

Количество контролируемых узлов может достигать одной тысячи на одну производственную площадку. Каждый узел может генерировать от нескольких десятков до нескольких сотен параметров.

Частота обновления параметров составляет от одной секунды до одной минуты в зависимости от критичности параметра.

Требования к производительности

Задержка доставки обновлений состояния до клиента не должна превышать двухсот миллисекунд от момента получения данных сервером до момента отображения на клиенте. Это требование обеспечивается использованием Socket.io/

Время ответа REST API для типовых запросов должно составлять менее пятисот миллисекунд для девяноста пяти процентов запросов.

11. Качество

Обеспечивается автоматическое переподключение Socket.io при обрыве связи с восстановлением состояния сессии. Клиентская библиотека Socket.io автоматически пытается восстановить соединение при его потере.

Аутентификация и авторизация реализованы через JWT-токены с ограниченным временем жизни. При успешной аутентификации пользователь получает токен, который должен передаваться в заголовках всех последующих запросов. Токен содержит информацию о пользователе и его роли.

Backend-часть может масштабироваться горизонтально путем запуска нескольких экземпляров приложения за балансировщиком нагрузки. Это позволяет распределять нагрузку и обеспечивать отказоустойчивость.

Socket.io поддерживает работу с несколькими серверами через адаптеры. Redis adapter обеспечивает синхронизацию событий и состояния комнат между различными экземплярами Socket.io серверов.